

①4 Basic Sorting Algorithms

Date: / / Page no: _____

□ Sorting :- Arrange in an order

Basic Sorting Algorithms:-

- Bubble Sort
- Selection Sort
- Insertion Sort
- Counting Sort (Advanced)

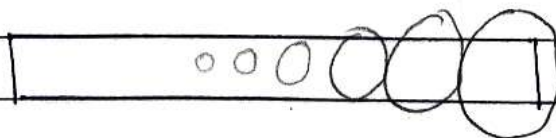
□ Bubble Sort:-

{5, 4, 1, 3, 2} — Unsorted Array

{1, 2, 3, 4, 5} — increasing order

{5, 4, 3, 2, 1} — Decreasing order

Large elements come to the end of array by swapping with adjacent elements.



Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.

In bubble sorting, the comparison of each element is accomplished with its adjacent element. If the first element is larger than the second one then the position of the elements are interchanged, otherwise not changed. The comparison process is repeated for all elements in the array. By this the largest element will be at last position.

Time complexity : $O(n^2)$

Advantages:-

1. Bubble Sort is easy to understand and implement.
2. It does not require any additional memory space.
3. It is a stable sorting algorithm, means that element with the same key value maintain their relative order in the sorted output.

Disadvantages:-

1. Bubble Sort has a time complexity of $O(n^2)$ which makes it very slow for large data set.

2. Bubble Sort is comparison based algorithm, thus the use of comparison operation can limit the efficiency of algorithm in certain cases.
3. It needs more number of swaps, which makes it slower.

Applications:-

1. As bubble Sort is Stable Sorting, it is used in critical applications, where stability is must.
2. Used in sorting small dataset, or nearly sorted arrays.

Example: —

5	10	2	3	1
0	1	2	3	4

→ Round 1

a[0] a[1]	5	10	2	3	1
a[1] a[2]	5	2	10	3	1
a[2] a[3]	5	2	3	10	1
a[3] a[4]	5	2	3	1	10

→ Round 2

a[0] a[1]	2	5	3	1	10
a[1] a[2]	2	3	5	1	10
a[2] a[3]	2	3	1	5	10

→ Round 3

$a[0]$	$a[1]$	2	3	1	5	10
$a[1]$	$a[2]$	2	1	3	5	10

→ Round 4

$a[0]$	$a[1]$	1	2	3	5	10
--------	--------	---	---	---	---	----

• Algorithm:-

- (i) Set $i = 0$
- (ii) Repeat step ③ to ⑦ while $(i < n-1)$
- (iii) Set $j = 0$
- (iv) Repeat step ⑤ to ⑦ while $(j < n-1-i)$
- (v) if $a[j] > a[j+1]$
then

$\text{int temp} = a[j]$
 $a[j] = a[j+1]$
 $a[j+1] = \text{temp}$
- (vi) Set $j = j+1 / j++$
- ~~(vii)~~ end of while of step (iv)
- (vii) Set $i = i+1 / i++$
end of while of step (ii)
- (viii) Exit.

Normal Bubble Sort
Worst = $O(n^2)$
Best = $O(n^2)$

In optimised sort
Worst = $O(n^2)$
Best = $O(n)$

Date: / / Page no: _____

→ Code :-

```
public class vj {
```

```
    public static void BSort (int vj[]) {
```

```
        for (int i=0; i < vj.length-1; i++) {  
            boolean swapped = false;
```

```
            for (int j=0; j < vj.length-1-i; j++) {  
                // in case of descending order
```

```
                if (vj[j] > vj[j+1]) {
```

```
                    int temp = vj[j];
```

```
                    vj[j] = vj[j+1];
```

```
                    vj[j+1] = temp; swapped = true;
```

```
                } if (swapped == false) { break; }
```

```
            }  
        }  
    }  
    public static void print (int vj[]) {
```

```
        for (int i=0; i < vj.length; i++) {
```

```
            s.op(vj[i] + " ");
```

```
        }  
        s.opLn();
```

```
    }  
    public static void main (String [] args) {
```

```
        int vj[] = {9, 3, 7, 5, 1, 2, 11};
```

```
        BSort(vj);  
        print(vj);  
    }  
}
```

Output:- 1 2 3 5 7 9 11

□ Selection Sort :-

Pick the smallest (from unsorted), put it at the beginning.

Selection Sort is comparison-based sorting algorithm. It sorts an array by repeatedly selecting the smallest (or largest) element from the unsorted portion and swapping it with the first ~~to~~ unsorted element. The process continues until the entire array is sorted.

Time Complexity = $O(n^2)$ - as there are two loops.

Advantages :-

1. Easy to understand and implement, making it ideal for teaching basic sorting concepts.
2. Requires only a constant $O(1)$ extra memory space.
3. It requires less number of swaps (or memory writes) compared to other algorithms except cycle sort.

* Selection, quick, heap $\rightarrow$ unstable.

Date: / / Page no: _____

Disadvantages:-

1. Higher time complexity, which makes it slower.
2. It is the unstable.

Applications:-

1. Perfect for teaching fundamentals of sorting.
2. Heap sort is based on selection sort.
3. Suitable for small list where the overhead of more complex algorithms, isn't justified and memory writing is costly and needs to be maintained.

Example: 30 10 80 5 20
 0 1 2 3 4

$\rightarrow$ Round 1

$a[0] \ a[1]$	10	30	80	5	20
$a[0] \ a[2]$	10	30	80	5	20
$a[0] \ a[3]$	5	30	80	10	20
$a[0] \ a[4]$	5	30	80	10	20

$\rightarrow$ Round 2

$a[1] \ a[2]$	5	30	80	10	20
$a[1] \ a[3]$	5	10	80	30	20
$a[2] \ a[4]$	5	10	80	30	20

$\rightarrow$ Round 3

$a[2] \ a[3]$	5	10	30	80	20
$a[2] \ a[4]$	5	10	20	80	30

$\rightarrow$ Round 4

$a[3] \ a[4]$	5	10	20	30	80
---------------	---	----	----	----	----

• Algorithm

1. Set $i = 0$;
2. Repeat step to while ($i < n - 1$)
3. Set $\text{minPos} = i$;
4. Set $j = i + 1$ & $\text{minPos} = j$
5. Repeat step to while ($j < n$)
6. if $a[\text{minPos}] > a[j]$ then
~~set $\text{minPos} = j$~~ set $\text{minPos} = j$
7. Set $j = j + 1 / j++$ end of while of step (5)
8. Then swap the elements :-

```
int temp = a[minPos];
a[minPos] = a[i]
a[i] = temp.
```

9. Set $i = i + 1 / i++$ end of while of step (2)
10. Exit.

Code :-

```
public class SelectionSort {
```

```
    public static void SSort(int vj[]) {
```

```
        for (int i = 0; i < vj.length - 1; i++) {
```

```
            int minPos = i;
```

```
            for (int j = i + 1; j < vj.length; j++) {
```


if (vj[minPos] > vj[j]) { ↗ in case of descending order sorting

minPos = j;

}

int temp = vj[minPos];

vj[minPos] = vj[i];

vj[i] = temp;

}

// with while loop

/*

int i = 0;

while (i < vj.length - 1) {

int minPos = i;

int j = i + 1;

while (j < vj.length) {

if (vj[minPos] > vj[j]) {

minPos = j;

}

j++;

}

```
int temp = vj[minPos];  
vj[minPos] = vj[i];  
vj[i] = temp;
```

```
    i++;  
}
```

```
public static void Print(int vj[]) {  
    for (int i=0; i<vj.length; i++) {  
        S.op. (vj[i] + " ");  
    }  
    S.o.pln();  
}
```

```
public static void main (String [] args) {  
    int vj[] = {9, 3, 7, 5, 1, 2, 12};  
    SSort(vj);  
    Print(vj);  
}
```

output :- 1 2 3 5 7 9 11

□ Differences b/w Selection Sort & Bubble Sort

Selection Sort	Bubble Sort
1. Select element and place it on correct position	Repeatedly swap adjacent elements.
2. Time complexity = $O(n^2)$ Space complexity = $O(1)$	Time complexity = $O(n^2)$ Space complexity = $O(1)$
3. Unstable - Not Stable	Stable.
4. lowest element placed first at its position	Highest element placed first at its position.
5. Less number of swap.	More Swaps.
6. This method is faster.	This method is slower.
7. It is an efficient Sorting technique.	It is not an efficient Sorting technique.

□ Insertion Sort :-

→ Inspiration → Cards
 → Idea → Pick an element (from unsorted parts) & place in the right position in sorted part.

Insertion Sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in an sorted portion of the list.

An insertion sort is one that sorts a set of elements by inserting element into an existing sorted file.

Suppose A is an array of n elements $A[1], A[2], \dots, A[n]$ in the memory. The insertion sort algorithm scans A from $A[1]$ to $A[n]$ and inserting each element $A[k]$ into its proper position in the previously sorted subarray $A[1], A[2], \dots, A[k]$.

Pass 1 $\rightarrow$ $A[1]$ by itself is sorted.

Pass 2 $\rightarrow$ $A[2]$ is inserted either before or after $A[1]$ so that $A[1], A[2]$ is sorted.

Pass 3 $\rightarrow$ $A[3]$ is inserted into its proper place i.e., before $A[1]$, between $A[1]$ & $A[2]$, or after $A[2]$ so that $A[1], A[2], A[3]$ is sorted.

Pass 4 $\rightarrow$ $A[4]$ is inserted into its proper place in $A[1], A[2], A[3]$ so that $A[1], A[2], A[3], A[4]$ is sorted.

...

Pass N $\rightarrow$ $A[n]$ is inserted into its proper place in $A[1], A[2], A[3], \dots, A[n-1]$ so that $A[1], A[2], A[3], \dots, A[n]$ is sorted.

Ex $a[] = \{23, 1, 10, 5, 2\};$

round 1 $\rightarrow$

23	1	10	5	2
----	---	----	---	---

 $\longrightarrow$

23	1	10	5	2
----	---	----	---	---

$r-2 \rightarrow$

23	1	10	5	2
----	---	----	---	---

 $\longrightarrow$

1	23	10	5	2
---	----	----	---	---

$r-3 \rightarrow$

1	23	10	5	2
---	----	----	---	---

 $\longrightarrow$

1	10	23	5	2
---	----	----	---	---

$r-4 \rightarrow$

1	10	23	5	2
---	----	----	---	---

 $\longrightarrow$

1	5	10	23	2
---	---	----	----	---

$r-5 \rightarrow$

1	5	10	23	2
---	---	----	----	---

 $\longrightarrow$

1	2	5	10	23
---	---	---	----	----

For writing algorithm, we need a temporary variable temp.

Algorithm:-

1. Set $i=1$
2. for $i=1$ to $(n-1)$ { $i < n$ }
3. Set $temp = arr[i]$
 $\&$ $prev = i-1$
4. Repeat step 4 & 5 while ($prev \geq 0$ & $arr[prev] > temp$)
5. Set
 $arr[prev+1] = arr[prev]$
 then $prev = prev-1$;
 end of the while loop

6. Set arr[prev + 1] = temp;
7. End of ~~for~~ for loop
8. Exit.

Code:-

```

public class InsertionSort {

    public static void InSort (int vj[]) {

        for (int i = 1; i < vj.length; i++) {

            int temp = vj[i];
            int prev = i - 1;

            while (prev >= 0 && vj[prev] > temp) {

                vj[prev + 1] = vj[prev];
                prev--;
            }

            vj[prev + 1] = temp;
        }
    }

    public static void Print (int vj[]) {

        for (int i = 0; i < vj.length; i++) {

            S.op. (vj[i] + " ");
        }

        S.op.ln();
    }
}

```



```
public static void main (String [] args) {  
    int vj[] = {9, 3, 7, 5, 1, 2, 11};  
    InsertSort (vj);  
    Print (vj);  
}
```

→ output :- 1 2 3 5 7 9 11

Time Complexity = $O(n^2)$

Auxiliary Space = $O(1)$

Advantages :-

1. Simple & easy to implement
2. Stable sorting algorithm
3. Efficient for small list and nearly sorted list.
4. Adaptive : The number of inversions is directly proportional to number of swaps.

Disadvantages

1. Inefficient for large lists.
2. Not as efficient as other sorting algorithms.
3. There are many swaps and comparison.

□ Inbuilt Sort:-

```
# import java.util.Arrays;
```

```
Array.sort(arr) // ascending order
```

Complexity = $O(n \log n)$

→ Suppose if we want to sort first 3 element from the Array of 5 element.

```
Array.sort(arr, si, ei)
```

- si = starting index

- ei = ending index // `Array.sort(arr, 0, 3);`

➤ For Descending order:-

```
import java.util.Collections;
```

```
Array.sort(arr, Collections.reverseOrder())
```

```
Array.sort(arr, si, ei, Collections.reverseOrder())
```

Note:- You need to set int (primitive data type) to Integer (object type variable) to use Collections.

→ Complexity of CS = $O(n + \text{range})$

Date: / / Page no: _____

□ Counting Sort :-

Counting Sort is a non-comparison-based sorting algorithm. It is particularly efficient when the range of input values is small compared to the number of elements to be sorted.

The basic idea behind counting sort is to count the frequency of each distinct element in the input array and use that information to place the element in their correct sorted positions.

The important thing to notice is that the range of input element is small and comparable to the size of the array.

1, 4, 1, 3, 2, 4, 3, 7

minimum = 1 } Range is minimum
maximum = 7

0	2	1	2	2	0	0	1
0	1	2	3	4	5	6	7

 → Count array (freq of element)

1	1	2	3	3	4	4	7
---	---	---	---	---	---	---	---

 → Original Array
~~1 4 1 3 2 4 3 7~~

Code: -

```
public class r{
```

```
    public static void CSort (int arr []) {
```

```
        int largest = Integer.MIN_VALUE;
```

```
        for (int i=0; i < arr.length; i++) {
```

```
            largest = Math.max (largest, arr[i]);
        }
```

```
        // new array
```

```
        int count [] = new int [largest + 1];
```

```
        for (int i=0; i < arr.length; i++) {
```

```
            count[arr[i]]++;
        }
```

```
        // Sorting :-
```

```
        int j = 0;
```

```
        for (int i=0; i < count.length; i++) {
```

```
            while (count[i] > 0) {
```



```
        arr[i] = i;  
        j++;  
        count[i] --;  
    }  
}  
  
public static void Print(int arr[]) {  
    for (int i=0; i<arr.length; i++) {  
        s.op (arr[i] + " ");  
    }  
}  
  
public static void main (String [] args) {  
    int arr[] = {5, 4, 1, 1, 3, 2};  
    CSort (arr);  
    Print (arr);  
}
```

→ Output:- 1 1 2 3 4 5